

Exploiting Wasserstein GANs, with Gradient Penalty, for Super Resolution of a Limited Dataset of Chemical Images

Gianmaria Pizzo

872966@stud.unive.it

Abstract

Single-image super-resolution (SR) aims at reconstructing a high-resolution (HR) image from a low-resolution (LR) one, while preserving semantic content and details. In this project, we focus on **chemical images** with very limited training data (less than 1200 examples), learning a mapping from 28×28 downsampled inputs to 64×64 outputs using a previously developed **Wasserstein GAN with Gradient Penalty (WGAN-GP)**. We combine the Wasserstein critic objective with a gradient penalty to stabilize training and mitigate common GAN pathologies (vanishing gradients, mode collapse). For what concerns the generator, we use a lightweight convolutional + upsampling architecture and optimize a mixture of **adversarial** and **pixel-wise L1** losses to trade-off between perceived sharpness and structural fidelity. We provide a training/testing pipeline with configuration management, TensorBoard logging, and evaluation via classical metrics (MSE/RMSE/PSNR/SSIM) and perceptual metrics (LPIPS/FID).

1 Introduction

Super-resolution (SR) is a fundamental inverse problem in computer vision: given an LR image, the goal is to recover an HR image that is consistent with the LR observation and lies on the manifold of realistic images. Formally, given an LR sample $x \in \mathbb{R}^{C \times H \times W}$ produced by a degradation operator $\mathcal{D}$ from an unknown HR image $y \in \mathbb{R}^{C \times \hat{H} \times \hat{W}}$, SR seeks an estimator $\hat{y} = G(x)$ approximating y .

In this project, we focus on **chemical images**, where data can be scarce and details can be thin or low-contrast, but still meaningful. We downsample chemical images to 28×28 pixels and train a model to reconstruct 64×64 outputs while preserving content and structure. The central difficulty is that pixel-wise objectives alone often yield

smooth/blurry reconstructions, while **adversarial training can improve realism but is notoriously unstable**.

To address this, we adopt **WGAN-GP**: we replace the standard discriminator with a **critic** trained to approximate the Wasserstein-1 distance, and enforce a **1-Lipschitz** constraint via gradient penalty (Arjovsky et al., 2017) (Gulrajani et al., 2017). We implement a small SR generator and a convolutional critic producing scalar realism scores.

We further stabilize the generator behavior by combining adversarial loss with an L1 reconstruction loss, as also motivated by SR-GAN-style tradeoffs between fidelity and perceptual realism (Ledig et al., 2016).

Contributions

- A complete SR pipeline for limited-data chemical imaging using WGAN-GP, including configuration, logging, training, and testing modules.
- A lightweight generator/critic pair for $28 \times 28 \rightarrow 64 \times 64$ SR, with stable WGAN-GP optimization and a mixed generator loss.
- Quantitative evaluation via MSE/RMSE/PSNR/SSIM/LPIPS and distributional evaluation via FID, with automatic export of per-image benchmarks.

2 Motivation

Other than matching the course’s requirements, the main goal of this work is to assess the **adaptability** of a previously developed model, under more challenging and restrictive settings such as **data scarcity** and **domain-shift**.

In the original work, we easily trained the model over large datasets of medical images, which are

typically dense in details. Therefore, the pre-processing pipeline managed to downsample the original 64×64 images without losing many important details, and training had many samples available. This allowed us to focus mainly on ensuring convergent training, through the fine-tuning of hyperparameters (i.e. learning rate, critic rounds, etc ...) rather than changing the architecture or the data flow.

Here, even though the architecture is already suitable for the main task, we hoped to **put it under "stress-test" conditions** and without the use of transfer learning nor pretraining. Therefore, the main objective of our research is to understand how well can, the same design, operate under the following conditions:

- The domain shift given by the **exclusive use of chemical images** which depict **sharp microscopic-like details** and can be easily affected by the use of interpolation for up-sampling/down-sampling (present in both the pre-processing process and the up-sampling architecture of the generator).
- The threatening scarcity of the data, and the consequences it can have on the tuning of training-related parameters.
- No employment of data augmentation pipelines (besides the necessary ones) to enlarge the datasets.
- No modifications in the generator's loss, which could benefit from a stronger signal that takes into account the finer details and penalizes more the over-smoothing behavior.
- No further improvements to the architecture, besides the consequences of hyperparameters' tuning.

To sum up, the objective is not to propose a new architecture, but to observe to which extent, the reasonably simple model, can be pushed when domain assumptions become less favorable.

3 Related Work

Before diving into our work, we explore methods proposed by the current scientific literature to portray the state of the art and grasp how we have produced the current architecture.

4 Generative Adversarial Networks (GANs)

5 Issues with GAN

Although GANs has been shown to achieve great realistic image generation, their training is not trivial; The process is known to be slow and unstable, due to the following aspects

- **Nash Equilibrium:** Both models learn independently, so achieving it is hard and requires careful tuning.
- **Vanishing Gradient:** If Discriminator evolves to be too strong (converges too quickly), the gradient of the generator's loss vanishes and prevents learning ([Gulrajani et al., 2017](#)).
- **Mode Collapse:** GANs sometimes fail to generate diverse samples and collapse into producing only a few modes of the data distribution (e.g., a GAN trained on faces might generate only a few similar-looking faces) ([Gulrajani et al., 2017](#)).

This is where we first thought of using Wasserstein and Gradient Penalty!

5.1 Wasserstein GANs and Gradient Penalty

Standard GANs optimize divergences (e.g., Jensen-Shannon) and may suffer from vanishing gradients when the discriminator becomes too strong ([Goodfellow et al., 2014](#)). WGAN re-frames training by minimizing the Wasserstein-1 distance (Earth Mover's Distance) and uses a critic with a Lipschitz constraint ([Arjovsky et al., 2017](#)). WGAN-GP improves the Lipschitz enforcement by penalizing deviations of the gradient norm from 1 along interpolations between real and fake samples, leading to more stable optimization ([Gulrajani et al., 2017](#)). These properties are especially attractive in limited-data regimes, which is a key aspect of our chemical dataset setting.

However, it is not enough, because we also employ Pixel Loss.

5.2 Super-Resolution with Pixel Losses

Classical CNN-based SR methods often regress HR pixels from LR inputs with L1/L2 losses. These losses encourage fidelity but can over-smooth high-frequency details because multiple

plausible HR solutions exist for the same LR observation.

To improve this we would need to adapt the architecture with the following direction.

5.3 Adversarial Super-Resolution

SR-GAN introduced adversarial training for SR to produce perceptually convincing textures and edges (Ledig et al., 2016). Adversarial losses typically sharpen results but can introduce hallucinations and instability, especially with small datasets.

In our case, the model will not be enlarged to include these aspects, but will offer a solid baseline for future improvements.

6 Model

This section describes the learning system: the generator and critic models, their objectives, and how the final training losses are defined to balance realism and fidelity in the super-resolution task.

6.1 Task Formulation

The overall goal is to map low-resolution inputs (28×28) into super-resolved outputs (64×64), while preserving content and improving perceptual quality. More formally: given a low-resolution input image $\mathbf{x}_{LR} \in \mathbb{R}^{C \times 28 \times 28}$ and its corresponding high-resolution target $\mathbf{x}_{HR} \in \mathbb{R}^{C \times 64 \times 64}$, the generator G_θ produces a super-resolved image

$$\mathbf{x}_{SR} = G_\theta(\mathbf{x}_{LR}).$$

A critic D_ϕ assigns real-valued scores to images to approximate the Wasserstein distance between the real and generated distributions, and it is trained to score real HR images higher than generated SR images.

6.2 Generator Architecture

The generator is a shallow convolutional network that performs feature extraction at the LR scale and then upsamples and refines features to reach 64×64 . The use of explicit upsampling followed by convolutional refinement helps keep the architecture lightweight and reduces common artifacts associated with transposed convolutions. The final activation is tanh, producing outputs in $[-1, 1]$, consistent with the preprocessing normalization. Figure 1 depicts exactly this.

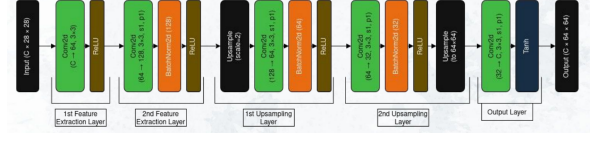


Figure 1: Generator Architecture

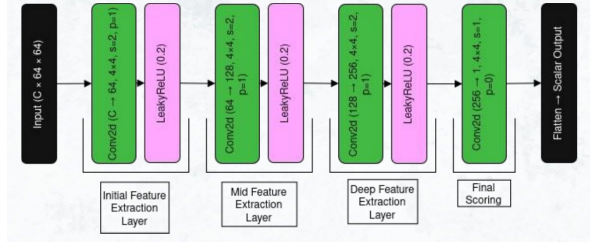


Figure 2: Critic Architecture

6.3 Critic (Wasserstein Discriminator) Architecture

The critic is a convolutional network with strided downsampling and LeakyReLU nonlinearities. It outputs unbounded real values (no sigmoid), as required by the Wasserstein formulation. Concretely, it applies a sequence of strided convolutions and then flattens the output to a 1D score vector, which is averaged in the loss computation. Its architecture is shown in figure 2.

6.4 Loss Functions

6.4.1 Critic Loss: Wasserstein Distance + Lipschitz Constraint

The critic objective is based on the Wasserstein-1 distance estimate:

$$\mathcal{L}_W = \mathbb{E}[D_\phi(\mathbf{x}_{SR})] - \mathbb{E}[D_\phi(\mathbf{x}_{HR})].$$

To satisfy the 1-Lipschitz constraint, the project supports two enforcement strategies: weight clipping (classic WGAN) or gradient penalty (WGAN-GP). In WGAN-GP, we sample interpolations

$$\hat{\mathbf{x}} = \epsilon \mathbf{x}_{HR} + (1 - \epsilon) \mathbf{x}_{SR}, \quad \epsilon \sim \mathcal{U}(0, 1),$$

and add the penalty

$$\mathcal{L}_{GP} = \mathbb{E}_{\hat{\mathbf{x}}} \left(\|\nabla_{\hat{\mathbf{x}}} D_\phi(\hat{\mathbf{x}})\|_2 - 1 \right)^2.$$

The final critic loss is:

$$\mathcal{L}_C = \lambda_W \mathcal{L}_W + \lambda_{GP} \mathcal{L}_{GP}.$$

This design stabilizes training by preventing abrupt critic gradients and is aligned with the WGAN-GP motivation described in the project material.

6.4.2 Generator Loss: Adversarial + Pixel-wise Fidelity

Since super-resolution must not only look realistic but also match the ground truth structure, the generator is optimized with a combination of:

$$\mathcal{L}_{adv} = -\mathbb{E}[D_\phi(G_\theta(\mathbf{x}_{LR}))], \quad \mathcal{L}_{L1} = \mathbb{E}[\|G_\theta(\mathbf{x}_{LR}) - \mathbf{x}_{HR}\|_1]$$

The final generator objective is:

$$\mathcal{L}_G = \lambda_{adv} \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_{L1}.$$

The adversarial term encourages perceptual realism (sharper details), while the L1 term anchors reconstructions to the target image, preserving layout and structure and mitigating hallucinations.

7 Methods

This section summarizes the configuration management for reproducible runs, and the dataset organization and preprocessing choices that define the SR learning.

7.1 Tools and Framework

The implementation is based on Python and PyTorch for model definition and training, with torchvision used for image transforms and grid utilities. We use TensorBoard to monitor losses, intermediate images, and metrics during training and validation. Quantitative evaluation relies on scikit-image for PSNR/SSIM/MSE, LPIPS for perceptual similarity, and pytorch-fid for dataset-level realism via FID. Configuration validation and structured access are handled through Pydantic.

7.2 Configuration Management

Hyperparameters and run settings are specified in a YAML file and parsed into a typed configuration object (AppConfig) composed of sub-configs (logging, data, preprocessing, training, validating, generator, critic). This provides validation, defaults, and consistent access to parameters such as batch size, learning rates, critic iterations, and loss weights. For reproducibility, each run stores a copy of the configuration inside its output folder and testing reloads the run-specific configuration.

7.3 Data Used and Data Management (High-Level)

The project supports two dataset families: MedicalMNIST-like datasets arranged in Train and Test subfolders, and an alternative 40mic

dataset with a similar split interface. Images are loaded from a flat folder structure and converted into paired LR/HR samples. Both LR and HR versions are derived from the same source image using deterministic resizing, grayscale conversion to the configured number of channels, and normalization to $[-1, 1]$ (mean/std 0.5), ensuring consistency with the generator output range.

8 Experimental Settings

This section describes the operational pipelines used in experiments: how configurations and datasets are instantiated, the training procedure (including validation), the evaluation/testing protocol, and the TensorBoard monitoring setup used to track learning dynamics and qualitative outputs.

8.1 Configuration Pipeline

Experiments are driven by a YAML configuration file specifying dataset selection, preprocessing flags, logging cadence, training schedule, and model hyperparameters. The training script loads the configuration, validates it through the Pydantic schema, and saves a snapshot of the effective configuration in the run folder. The testing script then loads this snapshot (runs/*i*/model/*i*/config.yaml) to guarantee the same settings (e.g., number of channels and dataset choice) are used during evaluation.

8.2 Data Pipeline

The data pipeline constructs paired $(\mathbf{x}_{LR}, \mathbf{x}_{HR})$ samples by applying two transform pipelines:

1. resize to LR size (e.g., 28×28) and HR size (e.g., 64×64),
2. grayscale conversion with `num_output_channels` equal to the configured channels,
3. conversion to tensor,
4. normalization to $[-1, 1]$ using mean/std 0.5 per channel.

For MedicalMNIST-like datasets, the training set is further split into train/validation subsets via a random split based on `train_percentage`. Otherwise, 40mic is simply loaded with a parallel, but more specific approach. The test loader uses batch size 1 for deterministic per-image evaluation and metric reporting.

8.3 Training Setup

We follow conservative optimization settings to reduce overfitting:

- Adam optimizer for both networks,
- small batch size (e.g., 16),
- small learning rates (e.g., $5 \cdot 10^{-5}$),
- WGAN-GP gradient penalty with tunable λ_{GP} and critic iterations c_iter .

These parameters are configured in the file `config.yaml` and loaded by the training script.

8.4 Training/Validation Pipeline

Training alternates multiple critic updates (as per definition) for each generator update (Goodfellow et al., 2014), controlled by `c_iter`. Each iteration

- computes critic scores on real HR images and generated SR images,
- applies either WGAN-GP (with gradient penalty) or classic WGAN (with weight clipping),
- and then updates the generator using the combined adversarial + L1 objective.

Validation is run every `validating.interval` epochs to compute quantitative metrics over the validation set and to log validation image grids. Model states and optimizer states are saved into a checkpoint file (`model.pth`) in the run directory, at the end of training and are ready to be used for testing.

8.5 TensorBoard Monitoring

TensorBoard is used as the central monitoring tool. First of all the project logs scalar curves for critic loss, generator loss, gradient penalty. In addition it records training-related diagnostics, and image grids. For what concerns the latter aspect, we include pictures for LR inputs, generated SR outputs, and HR targets at fixed intervals (`logging.interval`) for a limited number of images which usually matches the batch size. Logs are stored and organized into run-specific folders and subfolders (e.g., fake, real, metrics) to separate generated samples, real samples, and scalar metrics.

8.6 Testing Pipeline

Testing is performed by a dedicated script. It takes as input a run folder, and reloads the generator checkpoint along with the run-specific configuration, and evaluates the generator on the test set. For each test sample, the script saves the generated SR image and the corresponding HR target into `output/generated` and `output/original`. Furthermore, it computes per-image LPIPS, MSE, RMSE, PSNR, and SSIM, exports them to `benchmark.csv`. Finally computes test-set-level FID over the generated vs. real folders, and prints mean scores to standard output as a concise summary.

9 Results

9.1 Stability and Training Dynamics

The result is **stable training** compared to standard GAN or the simple use of Wasserstein distance with gradient clip. Empirically, the critic loss remains well-behaved and provides a usable learning signal for the generator over long runs, aided by multiple critic iterations per generator step. On the other hand, the generator struggles to reach (in less than 100 epochs) a balanced training experience.

9.2 Qualitative Behavior

The generator produces visually sound images and behaves particularly well when fed with images contain clear color cues. However, under the current settings, the model can **lose very fine details**.

9.3 Quantitative Metrics

In limited-data chemical SR, these metrics are useful but should be interpreted carefully:

Based on these numbers, let us see what the metrics say about our model.

- **Average fidelity is strong:** the scores of PSNR and SSIM suggest that, on average, the generated images are very close to the ground truth. These numbers typically indicate a good reconstruction quality.
- **Perceptual similarity is decent but uneven:** this confirms that the outputs are similar perceptually, but the model can sometimes produce pictures that look "close" by PSNR/SSIM but differ in texture/edges.

Table 1: Per-image quantitative metrics on the test set. Lower is better for MSE/RMSE/LPIPS, higher is better for PSNR/SSIM.

RMSE ↓	PSNR ↑	SSIM ↑	LPIPS ↓
0.0241	32.34	0.9410	0.0766
0.0375	28.51	0.9635	0.0631
0.0198	34.04	0.9282	0.1643
0.0458	26.78	0.9279	0.1182
0.0368	28.68	0.9357	0.0630
0.0125	38.01	0.9690	0.0437
0.0168	35.46	0.9723	0.0352
0.0236	32.52	0.9659	0.0651
0.0228	32.93	0.9549	0.0835
0.0252	31.94	0.9357	0.0854
0.0284	30.92	0.9462	0.1106
0.0210	33.19	0.9685	0.1240
0.025125	32.57	0.9506	0.0860

10 Discussion

A key lesson from this project is that **starting from a model choice** (GAN-first) rather than from a precise task analysis **can bias architectural** and pre-processing decisions.

Furthermore, as we mentioned, one important **source of smoothing lies in the process used to produce LR inputs**. Of course simple bilinear downsampling is fast and stable, but it often suppresses sharp edges. Like the SR-GAN paper suggests (Ledig et al., 2016), we should be either using some deeper architecture or modify the loss to include a perspective that really pushes the generator to capture more details (for example VGG + perceptual loss).

In fact, since also the **generator relies on interpolation to upsample**, the whole pipeline targets overly smooth reconstructions. Trivially, the magnitude of this aspect was less problematic for medical-style imaging, but is not acceptable for the current circumstances. Therefore, the **generator is mainly dominated by the critic**, as the former cannot keep up by construction (it is always penalized by the previously mentioned points).

Overall, our model offer **reliable reconstruction** (high SSIM) and achieves **good pixel-level accuracy** (PSNR 32.6 dB), which indicates that the L1 term is effectively anchoring the solution. However, the **spread in LPIPS** and the presence of hard outliers suggest that fine details remain challenging, but consistent with an interpolation-

heavy SR generator and a limited dataset.

Finally, the low-data regime amplifies all these problems: the generator tends to underfit rare high-frequency patterns (the fine details), and the adversarial training becomes even harder to balance, as changes in hyperparameters translate into less predictable training behaviors.

11 Conclusions

We presented a simple end-to-end SR framework and tested it on chemical images using WGAN-GP, demonstrating that Wasserstein objectives plus gradient penalty yield stable adversarial training in a low-data SR setting, and that the original model can still be robust even when we change the domain.

12 Future work

Motivated by the project conclusions and the SR literature (Ledig et al., 2016), the most useful future extensions are:

- **Up/Down-sampling Approach:** Changing the way we up/down-sample images in order to avoid putting the whole framework in a risky position.
- **Perceptual losses:** Adding a VGG-based feature loss to enforce fine detail recovery.
- **Stronger generator:** Modifying the current architecture to include residual blocks, and using smaller filters.
- **Data strategy:** Employing data augmentation strategies to enlarge smaller datasets and improve training.
- **Adaptive balance:** Using adaptive hyperparameters to regulate training based on the fragile balance between generator and critic learning.

References

- Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. **Wasserstein generative adversarial networks**. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. **Generative adversarial nets**. In *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc.

Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C. Courville. 2017. [Improved training of wasserstein gans](#). *CoRR*, abs/1704.00028.

Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew P. Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, and Wenzhe Shi. 2016. [Photo-realistic single image super-resolution using a generative adversarial network](#). *CoRR*, abs/1609.04802.